APRS Implementation Checklist

Legend

- [ ] [ ] Not started
- [x] [x] Completed

---

1. Project Setup & Configuration

1.1 Database Migration (MongoDB → PostgreSQL/Neon)

- [x] [x] Install Drizzle ORM + pg driver (`drizzle-orm`, `@neondatabase/serverless`, `drizzle-kit`)
- [x] [x] Remove mongoose dependency
- [x] [x] Create Drizzle config (`drizzle.config.js`)
- [x] [x] Create DB connection module (`server/src/config/db.js`)
- [x] [x] Define all schemas in `server/src/db/schema/`
- [x] [x] Run initial migration with `drizzle-kit push`

1.2 Cloudflare R2 Setup

- [x] [x] Install `@aws-sdk/client-s3` and `@aws-sdk/s3-request-presigner`
- [x] [x] Create R2 client config (`server/src/config/r2.js`)
- [x] [x] Replace multer disk storage with R2 upload
- [x] [x] Create file upload/download/delete utilities
- [x] [x] Generate presigned URLs for downloads

1.3 Environment Configuration

- [x] [x] Create `server/.env.example` with all required vars
- [x] [x] Create `client/.env.example` with VITE_ prefixed vars
- [x] [x] Create real `server/.env` (gitignored)
- [x] [x] Create real `client/.env` (gitignored)
- [x] [x] Update `.gitignore` for new patterns

1.4 Dependency Management

- [x] Update root `package.json` scripts (use pnpm)
- [x] Update `server/package.json` (add new deps, remove mongoose)
- [x] Update `client/package.json` (add recharts for radar charts)
- [x] Install all packages with pnpm

---

2. Database Schema Design

2.1 Tables

- [x] `users` table (id, username, email, password_hash, role, branch, prn, mobile, year, avatar_url, reated_at, updated_at)
- [x] `projects` table (id, unique_project_id, title, description, domain_tags, student_id, mentor_id, status, isibility, stars, created_at, updated_at)
- [x] `project_phases` table (id, project_id, phase_number, phase_name, completed, completed_at, escription)
- [x] `project_files` table (id, project_id, filename, original_name, r2_key, file_size, file_type, ploaded_at)
- [x] `project_members` table (id, project_id, user_id, role_in_group)
- [x] `feedback` table (id, project_id, reviewer_id, rating, comment, rubric_scores_json, created_at)
- [x] `notifications` table (id, user_id, title, message, type, read, created_at)
- [x] `refresh_tokens` table (id, user_id, token_hash, expires_at, created_at)

2.2 Indexes

- [x] Index on users.email (unique)
- [x] Index on users.prn (unique, nullable)
- [x] Index on projects.student_id
- [x] Index on projects.unique_project_id (unique)
- [x] Index on projects.status
- [x] Index on project_members.project_id
- [x] Index on feedback.project_id
- [x] Index on notifications.user_id
- [x] Index on refresh_tokens.token_hash

---

3. Server Implementation

3.1 Security Middleware

- [x] Install and configure `helmet` (HTTP security headers)
- [x] Install and configure `express-rate-limit` (global + per-route)
- [x] Install and configure `cors` with proper origin whitelist
- [x] Install `zod` for request validation
- [x] Create JWT middleware (`auth.middleware.js`) with access/refresh tokens
- [x] Create role-based authorization middleware (`role.middleware.js`)
- [x] Create request validation middleware (`validate.middleware.js`)
- [x] Add request body size limits

3.2 Authentication API

- [x] `POST /api/auth/signup` — register with validation (email, password strength, role)
- [x] `POST /api/auth/login` — authenticate, return access + refresh tokens
- [x] `POST /api/auth/refresh` — refresh access token using refresh token
- [x] `POST /api/auth/logout` — invalidate refresh token
- [x] `POST /api/auth/forgot-password` — send reset email
- [x] `POST /api/auth/reset-password/:token` — reset password via token
- [x] `PATCH /api/auth/update-password` — change password (authenticated)

3.3 User/Profile API

- [x] `GET /api/users/me` — get current user profile
- [x] `PATCH /api/users/me` — update profile fields (name, mobile, skills, avatar)
- [x] `GET /api/users/:id` — get user by ID (teachers/admin only)

3.4 Projects API

- [x] `POST /api/projects` — create project with file upload to R2
- [x] `GET /api/projects` — list with pagination, filter (status, branch, domain, visibility)
- [x] `GET /api/projects/:id` — get full project detail with phases, files, members
- [x] `PATCH /api/projects/:id` — update project (owner or teacher)
- [x] `DELETE /api/projects/:id` — delete project (owner only, cascades files)
- [x] `PATCH /api/projects/:id/status` — update status (teacher/admin only)
- [x] `PATCH /api/projects/:id/phase` — update single phase (owner)
- [x] `PATCH /api/projects/:id/phases` — update multiple phases (owner)
- [x] `GET /api/projects/student/:studentId` — student's projects
- [x] `GET /api/projects/search` — full-text search across departments

3.5 Students API

- [x] `GET /api/students` — list all students (teachers/admin)
- [x] `GET /api/students/branch/:branch` — students by branch with project info
- [x] `GET /api/students/:id/skills` — student skill radar data derived from projects

3.6 Feedback API

- [x] `POST /api/feedback` — submit feedback on project (teacher/expert)
- [x] `GET /api/feedback/project/:projectId` — get all feedback for a project

3.7 Notifications API

- [x] `GET /api/notifications` — get user's notifications (paginated)
- [x] `PATCH /api/notifications/:id/read` — mark notification as read
- [x] `PATCH /api/notifications/read-all` — mark all as read

3.8 Analytics API

- [x] `GET /api/analytics/skills/:userId` — radar chart data for student
- [x] `GET /api/analytics/department/:branch` — department-level stats
- [x] `GET /api/analytics/top-students` — top students by domain (industry filter)

3.9 File Management API

- [x] `POST /api/files/upload` — upload to R2, return file metadata
- [x] `GET /api/files/:key` — generate presigned download URL
- [x] `DELETE /api/files/:key` — delete file from R2 (owner/admin)

---

4. Client Implementation

4.1 Auth Context & Protected Routes

- [x] Create `AuthContext` with JWT token management
- [x] Auto-refresh token on expiry (interceptor)
- [x] Create `ProtectedRoute` component with role check
- [x] Persist auth state in localStorage (tokens only)
- [x] Redirect unauthenticated users to login

4.2 API Client

- [x] Create axios instance with base URL from env
- [x] Add request interceptor for auth headers
- [x] Add response interceptor for 401 → refresh flow
- [x] Centralize API calls in `services/` folder

4.3 Pages — Auth

- [x] Login page — connects to real API
- [x] Signup page — connects to real API with validation
- [x] Forgot Password page — connects to real API
- [x] Reset Password page — connects to real API

4.4 Pages — Common

- [x] Landing Page — functional with real navigation
- [x] Role Selection — navigates to correct login
- [x] Branch Selection — fetches real branch data
- [x] Profile Page — loads/updates real user data
- [x] Help Page — static content, functional

4.5 Pages — Student Dashboard

- [x] Project upload form submitting to real API + R2
- [x] Phase tracking with real phase update API
- [x] Project list from real API
- [x] Skill resources section
- [x] Progress analytics with real data

4.6 Pages — Teacher Dashboard

- [x] Branch selection loads real students
- [x] Student tracker table with real data
- [x] Project review queue from real API
- [x] Status update (approve/revise) via real API
- [x] Feedback submission via real API

4.7 Pages — Industry Expert Dashboard

- [x] Project catalog with real API filters (branch, domain, status)
- [x] Search across departments
- [x] Project evaluation modal with feedback API
- [ ] Top students by domain filter

4.8 Charts & Analytics

- [x] Install recharts
- [x] Radar/spider chart component for skill profiles
- [ ] Bar chart for department statistics
- [ ] Progress charts for project phases

---

5. API Testing Checklist

5.1 Auth Endpoints

- [x] POST /api/auth/signup — valid data → 201
- [x] POST /api/auth/signup — duplicate email → 400
- [x] POST /api/auth/signup — missing fields → 400
- [x] POST /api/auth/signup — weak password → 400
- [x] POST /api/auth/login — valid credentials → 200 + tokens
- [x] POST /api/auth/login — wrong password → 400
- [x] POST /api/auth/login — non-existent email → 400
- [x] POST /api/auth/login — mismatched role → 403
- [x] POST /api/auth/refresh — valid refresh token → 200 + new tokens
- [x] POST /api/auth/refresh — expired token → 401
- [x] POST /api/auth/refresh — invalid token → 401
- [x] POST /api/auth/logout — valid → 200
- [x] POST /api/auth/forgot-password — existing email → 200
- [x] POST /api/auth/forgot-password — non-existent email → 200 (same response)
- [x] POST /api/auth/reset-password/:token — valid token → 200
- [x] POST /api/auth/reset-password/:token — expired token → 400
- [x] PATCH /api/auth/update-password — valid → 200
- [x] PATCH /api/auth/update-password — wrong current password → 400
- [x] PATCH /api/auth/update-password — unauthenticated → 401

5.2 User Endpoints

- [x] GET /api/users/me — authenticated → 200 + user data
- [x] GET /api/users/me — unauthenticated → 401
- [x] PATCH /api/users/me — update name → 200
- [x] PATCH /api/users/me — update mobile → 200
- [x] GET /api/users/:id — as teacher → 200
- [x] GET /api/users/:id — as student (not self) → 403

5.3 Project Endpoints

- [x] POST /api/projects — valid data + files → 201
- [x] POST /api/projects — missing required fields → 400
- [x] POST /api/projects — unauthenticated → 401
- [x] POST /api/projects — as teacher → 403
- [x] GET /api/projects — as teacher → 200 + list
- [x] GET /api/projects — pagination (page, limit) → correct subset
- [x] GET /api/projects — filter by status → correct results
- [x] GET /api/projects — filter by branch → correct results
- [x] GET /api/projects/:id — exists → 200 + full detail
- [x] GET /api/projects/:id — not found → 404
- [x] PATCH /api/projects/:id — owner updates → 200
- [x] PATCH /api/projects/:id — non-owner → 403
- [x] DELETE /api/projects/:id — owner deletes → 200
- [x] DELETE /api/projects/:id — non-owner → 403
- [x] PATCH /api/projects/:id/status — teacher approves → 200
- [x] PATCH /api/projects/:id/status — student tries → 403
- [x] PATCH /api/projects/:id/status — invalid status → 400
- [x] PATCH /api/projects/:id/phase — valid phase update → 200
- [x] PATCH /api/projects/:id/phase — invalid phase name → 400
- [x] GET /api/projects/student/:studentId — valid → 200
- [x] GET /api/projects/search?q=keyword — returns matches → 200

5.4 Student Endpoints

- [x] GET /api/students — as teacher → 200
- [x] GET /api/students — as student → 403
- [x] GET /api/students/branch/:branch — valid branch → 200
- [x] GET /api/students/branch/:branch — invalid branch → 200 + empty
- [x] GET /api/students/:id/skills — valid → 200 + radar data

5.5 Feedback Endpoints

- [x] POST /api/feedback — teacher submits → 201
- [x] POST /api/feedback — student submits → 403
- [x] POST /api/feedback — missing fields → 400
- [x] GET /api/feedback/project/:id — exists → 200 + list

5.6 Notification Endpoints

- [x] GET /api/notifications — authenticated → 200 + list
- [x] GET /api/notifications — unauthenticated → 401
- [x] PATCH /api/notifications/:id/read — valid → 200
- [x] PATCH /api/notifications/read-all — valid → 200

5.7 Analytics Endpoints

- [x] GET /api/analytics/skills/:userId — valid → 200 + radar data
- [x] GET /api/analytics/department/:branch — valid → 200 + stats
- [x] GET /api/analytics/top-students?domain=CyberSecurity → 200 + list

5.8 File Endpoints

- [x] POST /api/files/upload — valid file → 201 + metadata
- [x] POST /api/files/upload — file too large → 400
- [x] GET /api/files/:key — valid → 200 + presigned URL
- [x] GET /api/files/:key — not found → 404
- [x] DELETE /api/files/:key — owner → 200
- [x] DELETE /api/files/:key — non-owner → 403

5.9 Rate Limiting

- [x] Auth endpoints — max 10 requests/15min per IP
- [x] General API — max 100 requests/15min per IP
- [x] File upload — max 20 requests/15min per user
- [x] Verify 429 response when exceeded

5.10 Security

- [x] All endpoints return proper CORS headers
- [x] Helmet headers present in responses
- [x] JWT tokens expire properly
- [x] SQL injection attempts are blocked by parameterized queries
- [x] XSS payloads in inputs are sanitized
- [x] File uploads validate mime types
- [x] Presigned URLs expire after configured time

---

6. UI Testing Checklist

6.1 Landing Page

- [x] Page loads without errors
- [x] Login button navigates to role selection
- [x] Signup button navigates to role selection
- [x] Responsive on mobile, tablet, desktop
- [x] Dark/light theme toggle works

6.2 Role Selection

- [x] Three role cards displayed (Student, Teacher, Expert)
- [x] Clicking card navigates to correct login route
- [x] Back navigation works

6.3 Login Page

- [x] Form validation (empty fields, email format)
- [x] Correct error messages on failed login
- [x] Successful login redirects to correct dashboard
- [x] “Forgot password” link works
- [x] Theme toggle visible

6.4 Signup Page

- [x] All fields present (name, email, password, branch)
- [x] Password strength indicator
- [x] Duplicate email shows error
- [x] Successful signup redirects to login
- [x] Branch dropdown populated

6.5 Forgot/Reset Password

- [x] Email form submits and shows success message
- [x] Reset form with valid token works
- [x] Reset form with expired token shows error
- [x] Password confirmation validation

6.6 Student Dashboard

- [x] Project upload form with file attachment
- [x] File upload progress indicator
- [x] Phase tracking UI with checkmarks
- [x] Project list displays real projects
- [x] Skill resources section accessible
- [x] Analytics/progress charts render
- [x] Sidebar navigation works
- [x] Logout works and clears session

6.7 Teacher Dashboard

- [x] Branch selection shows all departments
- [x] Student tracker table loads real data
- [x] Clicking student shows detail modal
- [x] Review queue shows pending projects
- [x] Approve/Revision buttons update status
- [x] Feedback form submits correctly
- [x] Search/filter works

6.8 Industry Expert Dashboard

- [x] Project catalog loads with filters
- [x] Branch, domain, status filters work
- [x] Search across departments returns results
- [x] Project evaluation modal opens/closes
- [x] Feedback submission works
- [ ] Top students filter by domain

6.9 Profile Page

- [x] Shows current user data
- [x] Edit form updates profile
- [x] Password change modal works
- [ ] Avatar upload (if implemented)

6.10 Help Page

- [x] FAQ accordion works
- [x] All links functional
- [x] Contact information displayed

6.11 Cross-cutting UI

- [x] Dark/light theme works across all pages
- [x] Navigation consistent across pages
- [x] Error boundaries catch and display errors
- [x] Loading states shown during API calls
- [x] Empty states for no data
- [x] Toast notifications for success/error
- [x] 404 page for undefined routes